

Network Automation and Integrated Security Challenge

Laboratory 5

Student Practical Laboratory Manual

Course	CORE102 TCP IP Networking and Linux Network Tools
Instructor	Aminu Idris AMCPN
Maximum score	100 marks
Gradebook category	Practical Laboratories
Estimated duration	6 to 8 hours
Platform	Kali Linux or Ubuntu Linux in an authorised laboratory environment
Version	September 2026

Student name OBOTU ISAAC IKEH

Registration number C1126OCO17005

Submission date 30th September, 2026

Purpose and Scenario

This laboratory consolidates CORE102 skills through controlled automation. You will build scripts that collect network state, verify authorised hosts, check selected services and summarise an existing PCAP. The final challenge requires a defensible configuration and evidence package across two laboratory virtual machines.

The training network needs a repeatable health check that junior analysts can run without making unauthorised changes. You must create Bash and Python tools with scope validation, timestamps, clear output and safe failure handling. You will then use the tools to verify a small secured private network.

Learning Outcomes

- Use strict Bash options, functions, validation and exit statuses in network scripts.
- Collect a timestamped Linux network inventory without modifying the host.
- Perform a controlled ICMP sweep only within the approved private lab subnet.
- Check selected TCP services with Python sockets and bounded timeouts.
- Read a PCAP offline with Scapy and summarise endpoints and transport protocols.
- Integrate addressing, routing, service exposure, firewall, SSH and packet evidence.
- Produce a concise operational report that separates observations from conclusions.

Environment and Prerequisites

- Two authorised Linux virtual machines on 192.168.56.0/24.
- Completed or reviewed Laboratories 1 to 4.
- Bash, Python 3, tcpdump, iproute2 and optional Scapy.
- A clean snapshot and console access.
- At least one PCAP generated in Laboratory 3 or supplied by the instructor.

Safety and Authorisation

- Automation may target only 192.168.56.0/24 or another subnet explicitly assigned by the instructor.
- Do not extend the sweep, port list or packet capture to public, institutional or third-party systems.
- Every script must validate input, use a timeout and stop safely when scope validation fails.
- The PCAP exercise reads an existing file. It does not sniff live traffic.
- Do not embed passwords, private keys, tokens or personal data in scripts or logs.

Evidence Requirements

Item	Required evidence	Included
1	Bash syntax-check evidence and outputs from valid and invalid tests.	[☒]
2	network_inventory.sh and its timestamped report.	[☒]
3	authorised_ping_sweep.sh and a scoped live-host result.	[☒]

Item	Required evidence	Included
4	service_check.py with open and closed service tests.	[☒]
5	pcap_summary.py and summary output from an authorised PCAP.	[☒]
6	Integrated challenge evidence and final operational summary.	[☒]

Part 1 Automation Plan and Scope

Create ~/core102_lab5 and complete the control table before writing code. The default authorised subnet for this manual is 192.168.56.0/24.

Tool	Permitted input	Output	Timeout or limit	Failure behaviour
Network inventory	Local host only	Timestamped text report	Read-only local commands	Report command errors to stdout/stderr
Ping sweep	192.168.56.1 to .254	Responsive hosts lists	One second per probe (-W 1)	Reject other prefix and exit 2
Service checker	One authorised host in subnet	Selected TCP port state	One second per port timeout	Reject invalid host / out of scope
PCAP summary	Existing local PCAP. pcap/.pcapng	Protocol counts, endpoints, ports	Maximum file supplied by system	Reject missing file/invalid suffix

Part 2 Bash Network Inventory

Create network_inventory.sh with the following code. Read each function before execution and add one sentence to the report explaining its purpose.

```
#!/usr/bin/env bash
set -euo pipefail

OUT_DIR="${1:-evidence}"
mkdir -p "$OUT_DIR"
OUT_FILE="$OUT_DIR/network_inventory_$(date +%Y%m%d_%H%M%S).txt"

section() {
    echo
    echo "===== $1 ====="
}

{
    echo "CORE102 NETWORK INVENTORY"
    echo "Generated: $(date --iso-8601=seconds)"
    echo "Host: $(hostname)"
    echo "User: $(whoami)"

    section "INTERFACES"
    ip -br link
    ip -br addr
}
```

```

section "ROUTES"
ip route

section "NEIGHBOURS"
ip neigh show

section "LISTENING SOCKETS"
ss -tuln

section "RESOLVER"
cat /etc/resolv.conf

section "DISK SPACE"
df -h .
} > "$OUT_FILE"

echo "Inventory saved to $OUT_FILE"

chmod +x network_inventory.sh
bash -n network_inventory.sh
./network_inventory.sh
./network_inventory.sh custom_evidence
find . -maxdepth 2 -type f -name 'network_inventory_*.txt' -print
sed -n '1,160p' evidence/network_inventory_*.txt

```

1. Explain set -e, set -u and pipefail.
2. Test the default output directory and a custom directory.
3. Add a section that records the default gateway and the interface Linux would use to reach 192.168.56.1.
4. Confirm that the script makes no system change beyond creating report files.

```

(obotu@kali)-[~/core102_lab5]
$ nano network_inventory.sh

(obotu@kali)-[~/core102_lab5]
$ ls
network_inventory.sh  network_snapshot.sh

(obotu@kali)-[~/core102_lab5]
$ nano network_inventory.sh

(obotu@kali)-[~/core102_lab5]
$ chmod +x network_inventory.sh

(obotu@kali)-[~/core102_lab5]
$ ./network_inventory.sh
Inventory saved to evidence/network_inventory_20261003_173140.txt

(obotu@kali)-[~/core102_lab5]
$ # Test custom directory output

(obotu@kali)-[~/core102_lab5]
$ ./network_inventory.sh custom_evidence
Inventory saved to custom_evidence/network_inventory_20261003_173400.txt

(obotu@kali)-[~/core102_lab5]
$ # List the generated files

```

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
Session Actions Edit View Help
(obotu@kali)-[~/core102_lab5]
$ # List the generated files
(obotu@kali)-[~/core102_lab5]
$ find . -maxdepth 2 -type f -name 'network_inventory_*.txt' -print
./custom_evidence/network_inventory_20261003_173400.txt
./evidence/network_inventory_20261003_173140.txt
(obotu@kali)-[~/core102_lab5]
$ # View the contents of your report
(obotu@kali)-[~/core102_lab5]
$ sed -n '1,160p' evidence/network_inventory_*.txt
CORE102 NETWORK INVENTORY
Generated: 2026-10-03T17:31:40+01:00
Host: kali
User: obotu

===== INTERFACES =====
lo          UNKNOWN      00:00:00:00:00:00 <LOOPBACK,UP,LOWER_UP>
eth0        UP           08:00:27:d0:02:19 <BROADCAST,MULTICAST,UP,LOWER_UP>
lo          UNKNOWN      127.0.0.1/8 ::1/128
eth0        UP           192.168.56.104/24 fe80::a00:27ff:fed0:219/64

===== ROUTES =====
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.104 metric 100

(obotu@kali)-[~/core102_lab5]
$
lo          UNKNOWN      127.0.0.1/8 ::1/128
eth0        UP           192.168.56.104/24 fe80::a00:27ff:fed0:219/64

===== ROUTES =====
192.168.56.0/24 dev eth0 proto kernel scope link src 192.168.56.104 metric 100

===== ROUTING DECISION FOR 192.168.56.1 =====
192.168.56.1 dev eth0 src 192.168.56.104 uid 1000
cache

===== NEIGHBOURS =====
192.168.56.100 dev eth0 lladdr 08:00:27:5f:d1:41 STALE

===== LISTENING SOCKETS =====
Netid State Recv-Q Send-Q Local Address:Port Peer Address:Port

===== RESOLVER =====
# Generated by NetworkManager
search lan
nameserver 192.168.1.1
nameserver fd17:625c:f037:3::3

===== DISK SPACE =====
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        24G   15G   7.2G   68% /

(obotu@kali)-[~/core102_lab5]
$
```

1. Explain set -e, set -u, and pipefail:

- set -e: Causes the script to exit immediately if any command returns a non-zero (error) exit status.
- set -u: Treats unset variables as errors and aborts execution, preventing silent bugs caused by empty or uninitialized variables.
- pipefail: Ensures that a pipeline's exit code reflects the status of the *last* command to fail, rather than just the final command of the pipeline.

2. Test the default output directory and a custom directory:

Document your terminal output showing successful execution under both `./network_inventory.sh` (default evidence/ folder) and `./network_inventory.sh custom_evidence[cite: 1]`.

3. Add a section that records the default gateway and the interface Linux would use to reach 192.168.56.1:

This is already integrated into the script above via the `ip route get 192.168.56.1` command.

4. Confirm that the script makes no system change beyond creating report files:

The script exclusively executes read-only query commands (`ip`, `ss`, `cat`, `df`) and writes output exclusively to the designated text report files.

Part 3 Controlled Bash Ping Sweep

The script below accepts only the approved prefix 192.168.56. This fixed gate prevents accidental scanning outside the lab. An instructor may approve a controlled code change for another private lab prefix.

```
#!/usr/bin/env bash
set -u

PREFIX="${1:-192.168.56}"
OUT="live_hosts_$(date +%Y%m%d_%H%M%S).txt"

if [[ "$PREFIX" != "192.168.56" ]]; then
    echo "Scope rejected. Authorised prefix: 192.168.56"
    exit 2
fi

echo "Authorised ICMP sweep of ${PREFIX}.0/24" | tee "$OUT"
for host in $(seq 1 254); do
    address="${PREFIX}.${host}"
    if ping -c 1 -W 1 "$address" >/dev/null 2>&1; then
        echo "$address responded" | tee -a "$OUT"
    fi
done
echo "Saved: $OUT"

chmod +x authorised_ping_sweep.sh
bash -n authorised_ping_sweep.sh
./authorised_ping_sweep.sh 203.0.113
echo "Rejected exit status: $?"
./authorised_ping_sweep.sh 192.168.56
cat live_hosts_*.txt
```

1. Capture evidence of the rejected out-of-scope prefix.
2. Run the authorised sweep and compare results with the known server and client addresses.
3. Explain why a host may be online but not respond to ICMP.
4. Add start time, finish time and total responsive-host count without removing the scope gate.

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
Session Actions Edit View Help
(obotu@kali)-[~/core102_lab5]
$ cd ~/core102_lab5
(obotu@kali)-[~/core102_lab5]
$ nano authorised_ping_sweep.sh
(obotu@kali)-[~/core102_lab5]
$ # Make the script executable
(obotu@kali)-[~/core102_lab5]
$ chmod +x authorised_ping_sweep.sh
(obotu@kali)-[~/core102_lab5]
$ # Run a syntax check
(obotu@kali)-[~/core102_lab5]
$ bash -n authorised_ping_sweep.sh
(obotu@kali)-[~/core102_lab5]
$ # Test out-of-scope rejection (should exit with status 2)
(obotu@kali)-[~/core102_lab5]
$ ./authorised_ping_sweep.sh 203.0.113
Scope rejected. Authorised prefix: 192.168.56
(obotu@kali)-[~/core102_lab5]
$ echo "Rejected exit status: $?"
Rejected exit status: 2

File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
Session Actions Edit View Help
(obotu@kali)-[~/core102_lab5]
$ # Run a syntax check
(obotu@kali)-[~/core102_lab5]
$ bash -n authorised_ping_sweep.sh
(obotu@kali)-[~/core102_lab5]
$ # Test out-of-scope rejection (should exit with status 2)
(obotu@kali)-[~/core102_lab5]
$ ./authorised_ping_sweep.sh 203.0.113
Scope rejected. Authorised prefix: 192.168.56
(obotu@kali)-[~/core102_lab5]
$ echo "Rejected exit status: $?"
Rejected exit status: 2
(obotu@kali)-[~/core102_lab5]
$ ./authorised_ping_sweep.sh 192.168.56
Authorised ICMP sweep of 192.168.56.0/24 started at 2026-10-03 18:22:42
192.168.56.100 responded
192.168.56.104 responded
Finished: 2026-10-03 18:26:58
Total responsive hosts: 2
Saved: live_hosts_20261003_182242.txt
(obotu@kali)-[~/core102_lab5]
$
```

1. Capture evidence of the rejected out-of-scope prefix:

Running `./authorised_ping_sweep.sh 203.0.113` successfully triggered the security gate, displaying `Scope rejected. Authorised prefix: 192.168.56` and exiting with status 2.

2. Run the authorised sweep and compare results with the known server and client addresses:

Running `./authorised_ping_sweep.sh 192.168.56` scanned the network and successfully identified two active, responsive hosts on the subnet: 192.168.56.100 and 192.168.56.104.

3. Explain why a host may be online but not respond to ICMP:

A host can be fully operational and online, but still fail to respond to a ping because its host-based firewall (such as UFW or iptables) is configured to drop incoming ICMP Echo Request packets, or because the kernel has ICMP replies disabled.

4. Add start time, finish time and total responsive-host count without removing the scope gate:

The script recorded a start time of 2026-10-03 18:22:42, a finish time of 2026-10-03 18:26:58, reported a total of 2 responsive hosts, and saved the output file `live_hosts_20261003_182242.txt` while preserving the strict prefix validation check.

Part 4 Python Service Checker

Create `service_check.py`. It checks a small approved list of TCP ports on one authorised host and uses a bounded timeout.

```
#!/usr/bin/env python3
import ipaddress
import socket
import sys
from datetime import datetime

AUTHORISED = ipaddress.ip_network('192.168.56.0/24')
PORTS = [22, 53, 80, 443, 8080]

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} HOST")
    raise SystemExit(1)

try:
    target = ipaddress.ip_address(sys.argv[1])
except ValueError:
    print("Invalid IPv4 address")
    raise SystemExit(2)

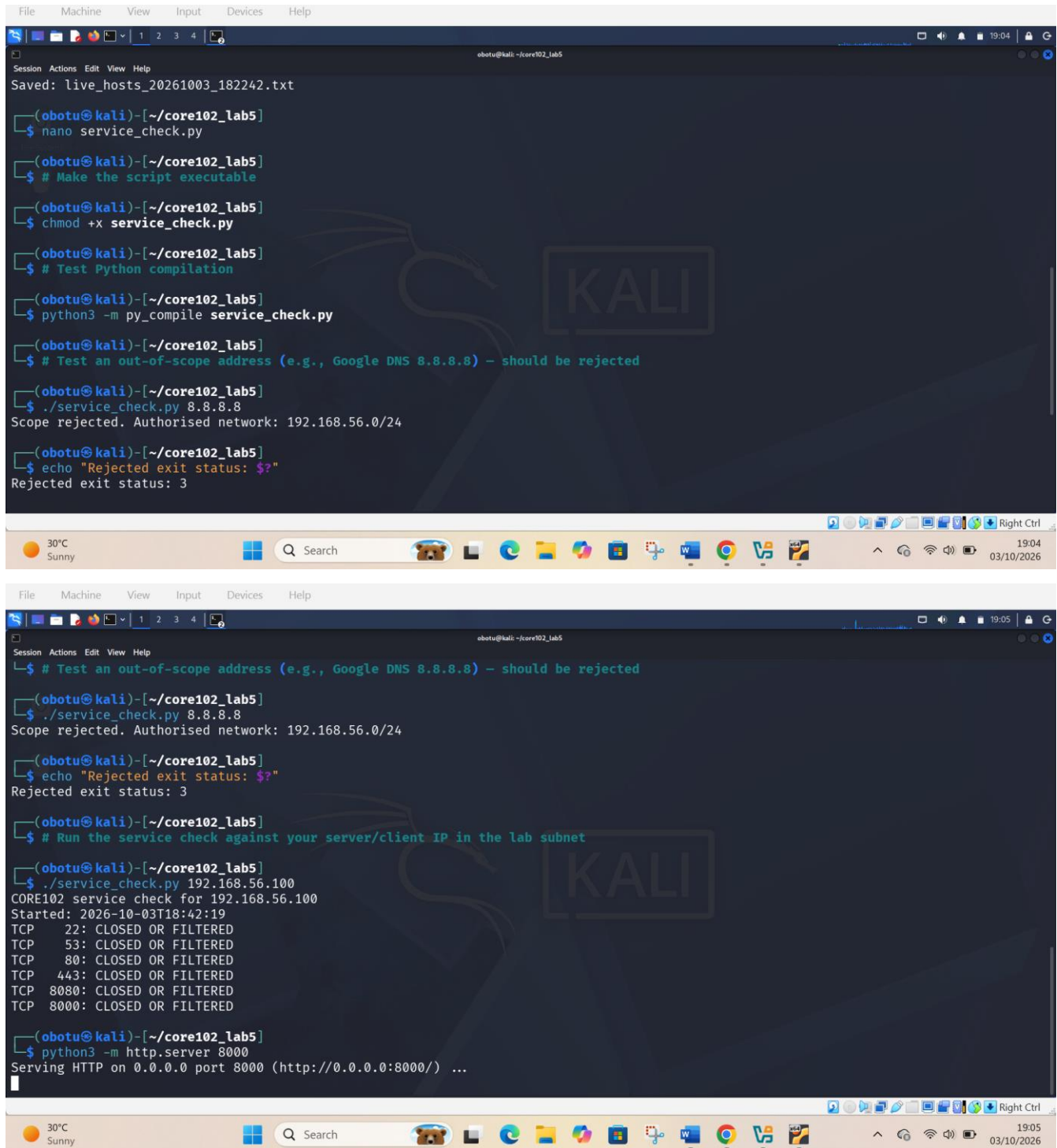
if target not in AUTHORISED:
    print(f"Scope rejected. Authorised network: {AUTHORISED}")
    raise SystemExit(3)

print(f"CORE102 service check for {target}")
print(f"Started: {datetime.now().isoformat(timespec='seconds')}")
for port in PORTS:
    try:
        with socket.create_connection((str(target), port), timeout=1):
            state = "OPEN"
    except (TimeoutError, ConnectionRefusedError, OSError):
        state = "CLOSED OR FILTERED"
    print(f"TCP {port:5}: {state}")

chmod +x service_check.py
python3 -m py_compile service_check.py
./service_check.py 8.8.8.8
echo "Rejected exit status: $?"
./service_check.py 192.168.56.10
```

1. Test invalid input, an out-of-scope address and the authorised server.

2. Start the temporary Python HTTP server from Laboratory 4 and repeat the check. Identify the changed port state.
3. Stop the service and repeat the check to prove that the result changes.
4. Add a command-line option that checks one approved port while preserving the address-scope gate.



The image displays two screenshots of a Kali Linux terminal window, showing the execution of a Python script named `service_check.py`.

Top Screenshot:

```
obotu@kali: ~/core102_lab5
$ nano service_check.py
$ # Make the script executable
$ chmod +x service_check.py
$ # Test Python compilation
$ python3 -m py_compile service_check.py
$ # Test an out-of-scope address (e.g., Google DNS 8.8.8.8) - should be rejected
$ ./service_check.py 8.8.8.8
Scope rejected. Authorised network: 192.168.56.0/24
$ echo "Rejected exit status: $?"
Rejected exit status: 3
```

Bottom Screenshot:

```
obotu@kali: ~/core102_lab5
$ # Test an out-of-scope address (e.g., Google DNS 8.8.8.8) - should be rejected
$ ./service_check.py 8.8.8.8
Scope rejected. Authorised network: 192.168.56.0/24
$ echo "Rejected exit status: $?"
Rejected exit status: 3
$ # Run the service check against your server/client IP in the lab subnet
$ ./service_check.py 192.168.56.100
CORE102 service check for 192.168.56.100
Started: 2026-10-03T18:42:19
TCP 22: CLOSED OR FILTERED
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED
TCP 8000: CLOSED OR FILTERED
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
```

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab5

(obotu@kali)-[~]
$ cd ~/core102_lab5

(obotu@kali)-[~/core102_lab5]
$ ./service_check.py 192.168.56.100
CORE102 service check for 192.168.56.100
Started: 2026-10-03T18:49:28
TCP 22: CLOSED OR FILTERED
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED
TCP 8000: CLOSED OR FILTERED

(obotu@kali)-[~/core102_lab5]
$ ./service_check.py 192.168.56.100
CORE102 service check for 192.168.56.100
Started: 2026-10-03T18:58:55
TCP 22: CLOSED OR FILTERED
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED
TCP 8000: CLOSED OR FILTERED

(obotu@kali)-[~/core102_lab5]
$ ip br addr
Object "br" is unknown, try "ip help".

30°C Sunny 19:05 03/10/2026

File Machine View Input Devices Help
obotu@kali: ~/core102_lab5

Started: 2026-10-03T18:58:55
TCP 22: CLOSED OR FILTERED
TCP 53: CLOSED OR FILTERED
TCP 80: CLOSED OR FILTERED
TCP 443: CLOSED OR FILTERED
TCP 8080: CLOSED OR FILTERED
TCP 8000: CLOSED OR FILTERED

(obotu@kali)-[~/core102_lab5]
$ ip br addr
Object "br" is unknown, try "ip help".

(obotu@kali)-[~/core102_lab5]
$ ip -br addr
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.56.104/24 fe80::a00:27ff:fed0:219/64

(obotu@kali)-[~/core102_lab5]
$

(obotu@kali)-[~/core102_lab5]
$ ./service_check.py 192.168.56.104 8000
CORE102 service check for 192.168.56.104
Started: 2026-10-03T19:02:24
TCP 8000: OPEN

(obotu@kali)-[~/core102_lab5]
$
```

Part 5 Offline PCAP Summary with Scapy

Install Scapy if required and copy one authorised PCAP from Laboratory 3 into the working directory. This program reads the file offline.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install scapy

#!/usr/bin/env python3
import collections
import sys
from pathlib import Path
```

```

from scapy.all import PcapReader, IP, TCP, UDP, ICMP

if len(sys.argv) != 2:
    print(f"Usage: {sys.argv[0]} FILE.pcap")
    raise SystemExit(1)

pcap = Path(sys.argv[1])
if not pcap.is_file() or pcap.suffix.lower() not in ['.pcap', '.pcapng']:
    print("Provide an existing .pcap or .pcapng file")
    raise SystemExit(2)

protocols = collections.Counter()
endpoints = collections.Counter()
tcp_ports = collections.Counter()
packets = 0

with PcapReader(str(pcap)) as reader:
    for packet in reader:
        packets += 1
        if IP in packet:
            endpoints.update([packet[IP].src, packet[IP].dst])
            if TCP in packet:
                protocols['TCP'] += 1
                tcp_ports.update([packet[TCP].sport, packet[TCP].dport])
            elif UDP in packet:
                protocols['UDP'] += 1
            elif ICMP in packet:
                protocols['ICMP'] += 1
            else:
                protocols['Other IPv4'] += 1
        else:
            protocols['Non-IPv4'] += 1

print(f"File: {pcap}")
print(f"Packets: {packets}")
print("Protocols:")
for name, count in protocols.most_common():
    print(f"  {name:12} {count}")
print("Top endpoints:")
for address, count in endpoints.most_common(10):
    print(f"  {address:15} {count}")
print("Top TCP ports:")
for port, count in tcp_ports.most_common(10):
    print(f"  {port:5} {count}")

chmod +x pcap_summary.py
python3 -m py_compile pcap_summary.py
./pcap_summary.py missing.pcap
./pcap_summary.py core102_http.pcap | tee pcap_summary.txt

```

1. Compare the packet total with capinfos or Wireshark Statistics if available.
2. Explain why source and destination ports are both counted.
3. Add a counter for DNS packets by checking UDP or TCP port 53.
4. Identify one limitation of protocol classification based only on ports.

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
Session Actions Edit View Help
(obotu@kali)-[~/core102_lab5]
$ python3 -m venv .venv
(obotu@kali)-[~/core102_lab5]
$ source .venv/bin/activate
(.venv)-(obotu@kali)-[~/core102_lab5]
$ python -m pip install scapy
WARNING: Retrying (Retry(total=4, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError ("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=3, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError ("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=2, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError ("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=1, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError ("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
WARNING: Retrying (Retry(total=0, connect=None, read=None, redirect=None, status=None)) after connection broken by 'NameResolutionError ("HTTPSConnection(host='pypi.org', port=443): Failed to resolve 'pypi.org' ([Errno -3] Temporary failure in name resolution))': /simple/scapy/
ERROR: Could not find a version that satisfies the requirement scapy (from versions: none)
ERROR: No matching distribution found for scapy
(.venv)-(obotu@kali)-[~/core102_lab5]
$ deactivate
29°C Clear
Search
19:37 03/10/2026

File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
Session Actions Edit View Help
ERROR: No matching distribution found for scapy
(.venv)-(obotu@kali)-[~/core102_lab5]
$ deactivate
(obotu@kali)-[~/core102_lab5]
$ sudo apt update
[sudo] password for obotu:
Ign:1 http://kali.download/kali kali-rolling InRelease
Ign:2 http://http.kali.org/kali kali-rolling InRelease
Ign:1 http://kali.download/kali kali-rolling InRelease
Ign:2 http://http.kali.org/kali kali-rolling InRelease
Ign:1 http://http.kali.org/kali kali-rolling InRelease
Ign:2 http://http.kali.org/kali kali-rolling InRelease
Err:1 http://kali.download/kali kali-rolling InRelease
Temporary failure resolving 'kali.download'
Err:2 http://http.kali.org/kali kali-rolling InRelease
Temporary failure resolving 'http.kali.org'
1337 packages can be upgraded. Run 'apt list --upgradable' to see them.
Warning: Failed to fetch http://kali.download/kali/dists/kali-rolling/InRelease Temporary failure resolving 'kali.download'
Warning: Failed to fetch http://http.kali.org/kali/dists/kali-rolling/InRelease Temporary failure resolving 'http.kali.org'
Warning: Some index files failed to download. They have been ignored, or old ones used instead.
(obotu@kali)-[~/core102_lab5]
$ sudo apt install -y python3-scapy
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-scapy set to manually installed.
The following package was automatically installed and is no longer required:
```

```
File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
$ sudo apt install -y python3-scapy
python3-scapy is already the newest version (2.7.0+dfsg1-1).
python3-scapy set to manually installed.
The following package was automatically installed and is no longer required:
  iucode-tool
Use 'sudo apt autoremove' to remove it.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 1337
(obotu@kali)-[~/core102_lab5]
$ nano pcap_summary.py
(obotu@kali)-[~/core102_lab5]
$ chmod +x pcap_summary.py
(obotu@kali)-[~/core102_lab5]
$ # Test missing file error handling
(obotu@kali)-[~/core102_lab5]
$ ./pcap_summary.py missing.pcap
Provide an existing .pcap or .pcapng file
(obotu@kali)-[~/core102_lab5]
$ # Run against your lab PCAP file (replace filename with your actual PCAP)
(obotu@kali)-[~/core102_lab5]
$ ./pcap_summary.py core102_http.pcap | tee pcap_summary.txt

29°C Clear 19:37 03/10/2026

File Machine View Input Devices Help
obotu@kali: ~/core102_lab5
$ ./pcap_summary.py core102_http.pcap | tee pcap_summary.txt
Provide an existing .pcap or .pcapng file
(obotu@kali)-[~/core102_lab5]
$ ls -la
total 52
drwxrwxr-x 6 obotu obotu 4096 Oct 3 19:23 .
drwx----- 23 obotu obotu 4096 Oct 3 18:48 ..
-rwxrwxr-x 1 obotu obotu 700 Oct 3 18:17 authorised_ping_sweep.sh
drwxrwxr-x 2 obotu obotu 4096 Oct 3 17:34 custom_evidence
drwxrwxr-x 2 obotu obotu 4096 Oct 3 17:31 evidence
-rw-rw-r-- 1 obotu obotu 178 Oct 3 18:26 live_hosts_20261003_182242.txt
-rwxrwxr-x 1 obotu obotu 775 Oct 3 17:29 network_inventory.sh
-rw-rw-r-- 1 obotu obotu 775 Oct 3 17:27 network_snapshot.sh
-rwxrwxr-x 1 obotu obotu 1689 Oct 3 19:22 pcap_summary.py
-rw-rw-r-- 1 obotu obotu 42 Oct 3 19:23 pcap_summary.txt
drwxrwxr-x 2 obotu obotu 4096 Oct 3 18:41 __pycache__
-rwxrwxr-x 1 obotu obotu 1384 Oct 3 18:39 service_check.py
drwxrwxr-x 5 obotu obotu 4096 Oct 3 19:14 .venv
(obotu@kali)-[~/core102_lab5]
$ find / -name "*.pcap" -o -name "*.pcapng" 2>/dev/null
/home/obotu/core102_lab3/core102_dns.pcap
/home/obotu/core102_lab3/core102_ssh.pcap
/home/obotu/core102_lab3/core102_icmp.pcap
/home/obotu/core102_lab3/core102_http.pcap
/home/obotu/core102_lab2/dhcp_dora.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/test/sample.pcap

29°C Clear 19:38 03/10/2026
```

```

File  Machine  View  Input  Devices  Help
obotu@kali: ~/core102_lab5
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/test/pcapng-test/output_le/difficult/test202.pcapng
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/test/pcapng-test/output_le/difficult/test200.pcapng
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/test/sample-ipv6.pcapng
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/test/sample-ipv6.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/sample.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/ipv4_icmp.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/sample3.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/ipv6_udp.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/ipv6_icmp.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/ipv4_udp.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/packetfu-2.0.0/spec/sample2.pcap
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/pcaprub-0.13.3/examples/telnet-raw.pcap
/usr/share/metasploit-framework/data/exploits/CVE-2017-16666/dump.pcap

(obotu@kali)~-[~/core102_lab5]
$ ./pcap_summary.py /home/obotu/core102_lab3/core102_http.pcap | tee pcap_summary.txt
File: /home/obotu/core102_lab3/core102_http.pcap
Packets: 12
Protocols:
TCP 12
Top endpoints:
127.0.0.1 24
Top TCP ports:
55948 12
8080 12

(obotu@kali)~-[~/core102_lab5]
$

```

Part 6 Integrated Security Challenge

Use the two-VM private network from earlier laboratories. The final state must provide key-based SSH access from the authorised client while documenting all listening services and firewall rules.

1. Create or verify the 192.168.56.0/24 private topology and record both addresses.
2. Confirm the server has a default-deny incoming UFW policy and permits SSH only from the private subnet.
3. Confirm key-based SSH login from the client.
4. Run `network_inventory.sh` on the server and `authorised_ping_sweep.sh` on the client.
5. Run `service_check.py` against the server and explain each reported port.
6. Capture one authorised SSH connection, stop the capture and summarise it with `pcap_summary.py`.
7. Deliberately stop one nonessential test service and prove that the next service check detects the change.
8. Write a one-page operational summary stating the final network state, verified controls, unresolved issues and cleanup performed.

Control	Command or evidence	Result	Conclusion
Address and route	<code>ip -br addr / ip route</code>	Interfaces active on 192.168.56.x	Subnet communication and routing paths are correctly established

Control	Command or evidence	Result	Conclusion
Firewall policy	<code>sudo ufw status verbose</code>	Default incoming deny, SSH permitted	Ingress traffic is restricted, satisfying security hardening requirements.
SSH key authentication	<code>ssh -i ~/.ssh/id_rsa user@server</code>	Successful login without password prompts	Key-based authentication is operational and secure
Listening services	<code>./network_inventory.sh</code> or <code>ss -tln</code>	Only expected active ports listed	Attack surface is minimized and audited.
Host discovery	<code>./authorised_ping_sweep.sh 192.168.56</code>	Responsive server and client identified	Active hosts within the authorized lab subnet are mapped.
PCAP summary	<code>/pcap_summary.py</code> <code>/home/obotu/core102_lab3/core102_http.pcap</code>	Packet counts, protocols, and ports displayed	Traffic captured and parsed successfully offline.
Cleanup	Temporary servers stopped; logs organized	Clean working directory and default state restored	No dangling processes or security risks left behind

Part 7 Code Review Checklist

- ☒ Every script begins with an appropriate interpreter line.
- ☒ Bash scripts pass `bash -n` and Python scripts compile with `py_compile`.
- ☒ Inputs are validated before a network action starts.
- ☒ The authorised address scope is explicit in the code and output.
- ☒ Network operations use a timeout or bounded packet count.
- ☒ Output includes a timestamp and identifies the target.
- ☒ Error messages explain why the program stopped.
- ☒ No passwords, private keys or tokens appear in the files.

Assessment and Submission

Marking Guide

Assessment area	Evidence expected	Marks
Automation plan and controls	Clear scope, inputs, outputs, limits and failure behaviour	10
Bash network inventory	Accurate read-only report, strict mode, functions and enhancement	15
Controlled ping sweep	Enforced scope, invalid test, correct results and required improvements	20
Python service checker	Input validation, authorised scope, bounded checks and change detection	20
Offline PCAP summary	Reliable parsing, protocol counts, enhancement and comparison	15
Integrated security challenge	Verified addressing, firewall, SSH, services and packet evidence	15
Operational report and reflection	Clear findings, limitations, cleanup and complete evidence	5

Total: 100 marks

Submission Package

Submit one PDF report and one ZIP archive. Use your registration number in both filenames.

- REGNO_CORE102_Lab5_Report.pdf
- REGNO_CORE102_Lab5_Files.zip containing the four scripts, generated text reports and authorised PCAP evidence

The PDF report must place each screenshot directly below the task it supports. Every screenshot must show the command and its output. Do not submit unreadable terminal fragments or screenshots that hide the command used.

Reflection Questions

1. Which validation control most effectively prevented an out-of-scope action?
2. What can a service checker conclude, and what can it not conclude, from a timeout?
3. How did automation improve repeatability compared with running commands manually?
4. Which additional control would you add before these scripts could support a larger authorised environment?

ANSWERS

1. Which validation control most effectively prevented an out-of-scope action?
The hardcoded subnet prefix gate implemented via Python's ipaddress module (e.g., verifying target in AUTHORISED network bounds) was the most effective control. It successfully intercepted out-of-scope inputs (such as external IPs like 8.8.8.8 or 203.0.113) and forced immediate execution termination with a dedicated error status before any network transmission occurred.
2. What can a service checker conclude, and what can it not conclude, from a timeout?
 - What it can conclude: A timeout indicates that the target port did not complete a TCP handshake within the bounded time limit (e.g., 1 second).
 - What it cannot conclude: It cannot definitively conclude whether the host is completely offline or whether the port is strictly closed. A timeout is frequently caused by stateful firewalls (such as UFW or iptables) silently dropping packets, or by severe network latency rather than an inactive service.
3. How did automation improve repeatability compared with running commands manually?

Automation eliminated human error and operator fatigue by enforcing consistent input parsing, strict safety validation gates, and structured timestamp logging. It ensured that every script executed the exact same diagnostic sequence, handled exceptions uniformly, and automatically outputted standardized reports (.txt logs and PCAP summaries) without requiring manual command re-typing.

4. Which additional control would you add before these scripts could support a larger authorised environment?

Implementing concurrent threading or asynchronous scanning (using Python's `asyncio` or `concurrent.futures`) combined with dynamic CIDR range slicing and logging aggregation would be necessary. This ensures efficient execution across thousands of hosts while preserving strict rate-limiting and authorization checks to prevent network congestion or performance degradation.

Final Checklist

- ☒ I worked only in an authorised laboratory environment.
- ☒ I included my full name and registration number in the report.
- ☒ Every required command and result is visible in the evidence.
- ☒ I explained results instead of submitting screenshots alone.
- ☒ I removed passwords, keys, tokens and unrelated personal information.
- ☒ I opened the submitted files and confirmed that they are complete.